Maximum Subarray Sum
using Kadane's Algorithm

Name	**:**	Rudra Pratap Singh
SAP ID	**:**	590032811
Batch	**:**	16
Semester	**:**	I

1. Problem Statement

Write a C program that reads the size and the elements of a variable-length integer array arr[]. The array must be passed, using its address, to a function named max_subarray_sum(). Pointers must be used to pass the array between the calling function (main) and max_subarray_sum().

The function max_subarray_sum() must:

- Find the maximum sum of any contiguous subarray of arr[] using Kadane's algorithm.
- Display the maximum subarray sum.
- If all elements are negative, display the largest (least negative) element.
- Ensure that the selected subarray contains at least one element.

2. Objective

The objective of this project is to understand and implement Kadane's algorithm, which solves the maximum subarray problem in a single pass over the array, i.e. in O(n) time and O(1) extra space. The project also demonstrates pointer-based array handling, pointer arithmetic, and dynamic memory allocation in C.

3. Concepts Used

- **Pointers:** the array is passed to the function as an int pointer holding the base address of the array.
- **Pointer arithmetic:** elements are accessed as *(arr + i) instead of arr[i].
- **Dynamic memory allocation:** malloc() creates an array of the size entered at run time, and free() releases it.
- **Functions:** the logic is placed in a separate function, max_subarray_sum().
- **Loops and conditions:** for loop and if-else are used in the algorithm.

4. Kadane's Algorithm

4.1 Idea

A subarray is a continuous part of an array. Kadane's algorithm scans the array from left to right and, at every index i, keeps track of two values:

- **`current_sum`** – the maximum sum of a subarray that ends exactly at index i.
- **`max_sum`** – the maximum sum found anywhere so far.

At each element there are only two choices: extend the previous subarray by adding the current element, or start a fresh subarray from the current element. Whichever gives the larger value is kept:

```
current_sum = max( arr[i] , current_sum + arr[i] )
```

4.2 Handling all-negative arrays

Both current_sum and max_sum are initialised with the first element arr[0] (and not with 0). Because of this, the chosen subarray always contains at least one element, and when every element is negative the algorithm correctly returns the largest (least negative) element.

4.3 Steps

1. Read the size n and allocate memory for n integers.
2. Read the n elements into the array using pointers.
3. Call max_subarray_sum(arr, n), passing the base address of the array.
4. Set current_sum = max_sum = *arr (the first element).
5. For i = 1 to n − 1: set current_sum to the larger of *(arr + i) and current_sum + *(arr + i).
6. If current_sum > max_sum, update max_sum = current_sum.
7. After the loop, display max_sum.
8. Free the allocated memory and end the program.

4.4 Complexity

Measure	Value	Reason
Time complexity	O(n)	Single pass over the array
Space complexity	O(1)	Only two extra variables

5. Dry Run (Example 1)

Array: `-2 1 -3 4 -1 2 1 -5 4` (n = 9). Initially current_sum = max_sum = −2.

Index i	arr[i]	current_sum = max(arr[i], current_sum + arr[i])	max_sum
0	−2	−2 (initial)	−2
1	1	max(1, −1) = 1	1
2	−3	max(−3, −2) = −2	1
3	4	max(4, 2) = 4	4
4	−1	max(−1, 3) = 3	4
5	2	max(2, 5) = 5	5
6	1	max(1, 6) = 6	6
7	−5	max(−5, 1) = 1	6
8	4	max(4, 5) = 5	6

Result: The maximum subarray sum is 6, given by the subarray 4, −1, 2, 1.

6. C Program

File name: max_subarray.c

6.1 Header files, prototype and main() function

```c
#include <stdio.h>
#include <stdlib.h>

/* Function prototype: the array is received through a pointer */
void max_subarray_sum(int *arr, int n);

int main(void)
{
    int n, i;
    int *arr;

    printf("Enter the size of the array: ");
    if (scanf("%d", &n) != 1 || n <= 0)
    {
        printf("Invalid size. Array must contain at least one element.\n");
        return 1;
    }

    /* Allocate memory for a variable-length array */
    arr = (int *)malloc(n * sizeof(int));
    if (arr == NULL)
    {
        printf("Memory allocation failed.\n");
        return 1;
    }

    printf("Enter %d elements: ", n);
    for (i = 0; i < n; i++)
    {
        if (scanf("%d", arr + i) != 1)
        {
            printf("Invalid input.\n");
            free(arr);
            return 1;
        }
    }

    /* Pass the address of the array to the function */
    max_subarray_sum(arr, n);

    free(arr);
    return 0;
}
```

6.2 max_subarray_sum() function

```
/* Finds and displays the maximum subarray sum (Kadane's algorithm) */
void max_subarray_sum(int *arr, int n)
{
    int i;
    int current_sum = *arr;  /* best sum of a subarray ending here  */
    int max_sum = *arr;      /* best sum so far (at least 1 element) */

    for (i = 1; i < n; i++)
    {
        /* Extend the previous subarray or start a new one at arr[i] */
        if (*(arr + i) > current_sum + *(arr + i))
            current_sum = *(arr + i);
        else
            current_sum = current_sum + *(arr + i);

        if (current_sum > max_sum)
            max_sum = current_sum;
    }

    printf("Maximum subarray sum: %d\n", max_sum);
}
```

7. Sample Outputs

Example 1

```
Enter the size of the array: 9
Enter 9 elements: -2 1 -3 4 -1 2 1 -5 4
Maximum subarray sum: 6
```

Explanation: the contiguous subarray with the maximum sum is 4 + (−1) + 2 + 1 = 6.

Example 2

```
Enter the size of the array: 6
Enter 6 elements: -8 -3 -6 -2 -5 -4
Maximum subarray sum: -2
```

Explanation: all elements are negative, so the largest element, −2, is selected.

Example 3

```
Enter the size of the array: 4
Enter 4 elements: 5 -2 3 4
Maximum subarray sum: 10
```

Explanation: the entire array has the maximum sum: 5 + (−2) + 3 + 4 = 10.

Additional Test Cases

Input array	Expected output	Program output
-7 (single element)	-7	-7
0 (invalid size)	Error message	Invalid size. Array must contain at least one element.

8. Conclusion

The program successfully finds the maximum contiguous subarray sum using Kadane's algorithm in a single pass. By initialising current_sum and max_sum with the first element,

the program always selects at least one element and handles arrays with all negative numbers correctly. The array is read into dynamically allocated memory and is passed to max_subarray_sum() through a pointer, with elements accessed using pointer arithmetic, as required. The program produced the correct output for all the given examples and the additional test cases.